Demo readiness and panel runbook

Group 15 • Final assessment preparation • 5 November 2026

The fresh report-preparation check found healthy API, scheduler, and both Grafana instances, 16 assigned partitions, zero application queue, and stopped local generators. Demo input was `experiment`, consumer group `group15-temperature-demo-v1`, and delivery used the external course endpoint. This dated record does not replace a check immediately before the exam.

Read-only checks before the professor starts

From the repository root:

```
curl --fail http://127.0.0.1:3003/health
curl --fail http://127.0.0.1:8001/health
curl --fail http://127.0.0.1:8001/status
sudo -n docker ps --format '{{.Names}} {{.Status}}'
df -h .
```

Inspect actual topic/group, receiver, assigned partitions, queue trend, and notification failures. The earlier broker watermark check found zero sampled lag; the status endpoint's application queue alone cannot prove that all retained broker work is consumed. A separate preflight can test course authentication with its own temporary consumer group:

```
.venv/bin/python scripts/check-course-kafka.py --topic experiment
```

That script neither publishes messages nor commits the live group's offsets. It confirms a temporary consumer's connection/assignment, not that the actual scheduler is processing correctly.

Read outbox state without printing payloads or credentials:

```
sudo -n docker compose --env-file .env.monitoring \
  -f compose.yml -f compose.demo.yml exec -T postgres \
  psql -U observability -d observability -c \
  "SELECT status,count(*) FROM notification_outbox GROUP BY status;"
```

A group member should also query the public API from their laptop using the VM address and a known real experiment ID. Localhost health does not establish the professor's firewall/routing path. Choose a real history time window and compare meaningful values; an empty array for an unobserved period is not a consistency demonstration.

Open the two local panels

Run this on your laptop with your existing SSH key/options:

```
ssh -N -L 3004:127.0.0.1:3004 -L 3009:127.0.0.1:3009 \
  ubuntu@YOUR_VM_PUBLIC_IP
```

Open the operational dashboard at `http://localhost:3004` and local instructor-style dashboard at `http://localhost:3009/d/cec-course-grading`. Use the existing Grafana credential setup described in MONITORING.md; do not paste private credentials into screenshots or reports.

View	What it demonstrates
Operational Grafana, 3004	Queue, delivery, cache, pool, CPU/disk, and cost observations
Local instructor-style Grafana, 3009	Original panels over local test truth and metrics; completed-run time range
Latest local test, bottom panel	Our extra explicit pass/fail checks for the selected saved run
Course-hosted Grafana	The course environment's own current observations; not implied by a local pass

The runner pins local Grafana to a completed test window. Inspect the time range and nonzero workload before interpreting green. `course_test_passed=1` is our local metric, not a professor-issued grade. Saved

benchmark JSON/PDFs remain evidence after live metrics age out under retention.

Keep demo and test modes distinct

Mode	Topic and destination
Professor demo	External `experiment`, existing demo consumer group, external notification API
Independent development	Local `experiment`, local notification sink
Original-tool self-test	Local course-test topic/group, original Rust notification validator

These modes share the application project/database. `make course-test`, `make course-stress`, and `make course-variable` change local deployment configuration and create workload; run them only in an intentional local-test window. `make course-stop` stops generators while preserving panels/results. It does not restore demo topic or receiver by itself.

If tests were running, stop them with:

```
make course-stop
```

Before changing receiver mode, check that pending jobs are understood and drained to the correct receiver. Preserve and investigate unexpected jobs; blindly deleting them or sending local validator jobs to the external endpoint breaks the evidence and contract.

Restore demo configuration when needed

This is an operational recovery action, not a step to execute against an already healthy professor test. Reuse existing database credentials and volumes. With the prepared `.env.monitoring` and course credential files present:

```
sudo -n docker compose --env-file .env.monitoring \
  -f compose.yml -f compose.demo.yml -f compose.monitoring.yml \
  up -d --wait postgres redis data executor scheduler notification api \
  prometheus node-exporter grafana
```

If images are absent, rebuild the application first using the documented deployment instructions. Do not change the consumer group merely to discard old input, and do not reset offsets to latest to hide lag. Inspect both broker retention and event age when recovering a long outage.

Local `kafka`, `notification-sink`, and `course-notifications` are test infrastructure; they can be stopped after a completed test window when returning to external demo mode. Preserve their named volumes and keep the local panel/database if you want to display archived results. Do not issue an indiscriminate command that kills all containers.

Disk incident and recovery policy

The user's cleanup removed containers/images but retained named volumes. Recovery reopened PostgreSQL with more than 2.3 million measurements and later processed retained Kafka backlog. The earlier outage's missed deadlines remain missed. Complete reconstruction of input expired from Kafka is not established.

Logs and local Kafka/Prometheus retention now have limits; disk alerts warn below 15% and become critical below 8%, with a separate projected-exhaustion alert. Active segments/WAL and persistent history mean retention settings are not exact total-disk caps. Infrastructure alerts are visible in Prometheus; external alert delivery has not been configured.

Inspect capacity before compiling tools or adding optional workloads. Preserve authoritative data; never free space by deleting files inside PostgreSQL's volume. A tested off-host backup/restore program remains future work. Disposable build cache can be removed after preserving the compiled instructor-tool image; source/image fingerprint reuse avoids a normal no-change test recompiling Rust. `--rebuild-tools`

intentionally forces a rebuild and may need substantial space.

A concise live demonstration

Show fresh health and actual demo mode, then a meaningful history query, current workload/alert evidence, and the bottleneck dashboard. Explain a failing historical capacity test as part of the measured design process. Be ready to open the exact JSON and the relevant code/test. All members should know how to stop local generators without stopping the actual service.

A current healthy service plus an old local green panel establishes two different facts. Only the professor's current evaluation establishes the result of that applied workload.